LAB6 REPORT

Name: Mourya Nandan

Roll No: CS23B006

QUESTION 1:

1. Initial Lab Setup

- **QEMU Emulator:**
  - The qemu-system-riscv64 package was installed on the host OS. This version of QEMU provides a "user-mode" networking stack that simulates a virtual LAN, placing the xv6 guest at IP 10.0.2.15 and the host at 10.0.2.22.
- **RISC-V Toolchain:**
  - A cross-compiler toolchain (gcc-riscv64-linux-gnu) was installed to compile the C and Assembly source code of xv6 into a RISC-V executable kernel.
- **Source Code:**
  - The project began by cloning the xv6-labs-2024 repository and switching to the net branch, which contained the skeleton code for the networking components.

2. The Lab's Objective

The primary goal was to implement a functional UDP networking subsystem within the xv6 kernel. This involved bridging the gap between the low-level network hardware and high-level user applications. The work was divided into two main components:

a. **Completing the E1000 Network Driver:**
   i. We were tasked with finishing the driver for the E1000 NIC. This involved implementing the logic to handle the transmission and reception of raw Ethernet frames using Direct Memory Access (DMA) descriptor rings.

b. **Implementing the Network Stack and System Calls:**
   i. We had to build the software layer that processes these raw frames. This was exposed to user applications via three new system calls[4]:
   - bind(short port): To reserve a UDP port for listening.
   - send(short sport, int dst, short dport, char *buf, int len): To construct and send a UDP packet.
   - recv(short dport, int *src, short *sport, char *buf, int maxlen): To receive a UDP packet, blocking the process if no packet is available.

3. Detailed Implementation and Code Changes

3.1 kernel/e1000.c: The Network Driver

- **Objective:**
  - To enable the kernel to exchange raw Ethernet frames with the simulated E1000 hardware.
- **Implementation Details:**
  - Packet Reception (e1000_recv):
    - This function was implemented to run on every network interrupt. It scans a circular array of receive descriptors (rx_ring) shared with the hardware. When a descriptor's "Descriptor Done" (DD) status bit is set by the hardware, the

function takes the associated buffer, passes it to the network stack via net_rx, and resets the descriptor for future use. It then updates the RDT (Receive Descriptor Tail) register to inform the hardware of the newly available descriptors.

- Packet Transmission (e1000_transmit):
  - This function sends a packet. It finds a free descriptor in the transmit ring (tx_ring), copies the packet data into a dedicated kernel buffer, programs the descriptor with the buffer's address and length, and updates the TDT (Transmit Descriptor Tail) register, signaling the hardware to begin transmission.

- **Challenges & Bugs (Deadlock Resolution):**
  - A panic: acquire error was encountered, indicating a deadlock. The trace revealed that e1000_recv (holding e1000_lock) would call arp_rx, which in turn called e1000_transmit, which tried to acquire the same e1000_lock again.
- **Solution**:
  - The transmit logic was refactored into two functions. e1000_transmit_locked was created to perform the transmission without locking, while e1000_transmit was converted into a simple wrapper that acquires the lock and calls the _locked version. The arp_rx function was then modified to call the non-locking e1000_transmit_locked directly, resolving the deadlock.

```
int
e1000_transmit_locked(char *buf, int len)
{
  if(!(tx_ring[tx_tail].status & E1000_TXD_STAT_DD)){
    return -1;
  }

  if(tx_bufs[tx_tail])
    kfree(tx_bufs[tx_tail]);

  tx_bufs[tx_tail] = kalloc();
  if(!tx_bufs[tx_tail]){
    return -1;
  }
  memmove(tx_bufs[tx_tail], buf, len);

  tx_ring[tx_tail].addr = (uint64)tx_bufs[tx_tail];
  tx_ring[tx_tail].length = len;

  tx_ring[tx_tail].cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;

  tx_tail = (tx_tail + 1) % TX_RING_SIZE;
  regs[E1000_TDT] = tx_tail;

  return 0;
}

int
e1000_transmit(char *packet, int len) {
  acquire(&e1000_lock);
  int r = e1000_transmit_locked(packet, len);
  release(&e1000_lock);
  return r;
}
```

3.2 kernel/net.c: The Network Stack and System Calls

- **Objective:**
  - To implement the logic for sockets, packet parsing, queuing, and the user-facing system calls.
- **Implementation Details:**
  - Data Structures: To manage network state, two new structs were created: struct pkt to represent a generic queued packet (containing a pointer to the data buffer and its length), and struct sock to represent a socket (containing the port number, a listening flag, and a queue of pkts).
  - Packet Routing (net_rx): This function became the central hub for incoming packets. It first determines the packet's type from the Ethernet header.
  - If it's an ARP packet, it delegates to the arp_rx function to handle the address resolution request.
  - If it's an IP packet, it parses the IP and UDP headers to find the destination port. It then searches the global sockets table for a matching, listening socket. If a match is found, the packet is appended to the socket's rx_queue, and wakeup is called to awaken any process sleeping on that socket.

- **Blocking Receive (sys_recv):**
  - The core of this system call is the sleep/wakeup mechanism. If a process calls recv and the socket's queue is empty, it sets a pointer to itself in the sock struct (s->proc = myproc()) and calls sleep(), which atomically releases the socket's lock and puts the process to sleep. When net_rx later queues a packet for this socket, it sees the stored process pointer and calls wakeup(), allowing sys_recv to resume.
- **Packet Sending (sys_send):**
  - This function constructs a full network packet in a kernel buffer. It builds the Ethernet, IP, and UDP headers, uses copyin to get the payload from the user, and then passes the completed packet to e1000_transmit.
- **Challenges & Bugs:**
  - ARP Handling: Initially, the tests hung because net_rx only handled IP packets. ARP requests (EtherType = 0x806) from the host were being silently dropped. The fix was to add logic to net_rx to recognize ARP packets and pass them to the arp_rx function, which then sends the required reply.
  - Incorrect Packet Length: Tests were failing with "wrong length" errors. This was because sys_recv was calculating the payload length from the total Ethernet frame size, which can include padding.
  - Fix: The calculation was changed to use the authoritative length from the IP header's ip_len field.
  - `payload_len = ntohs(iphdr->ip_len) - sizeof(struct ip) - sizeof(struct udp);`
  - Memory Leak: The free: FAILED test indicated a memory leak. The bug was in sys_send, which called kalloc() to create a buffer for each outgoing packet but never called kfree().
  - Fix: A kfree(buf) call was added at the end of sys_send, after the packet data had been passed to the driver.

4. The Testing Workflow

Testing required a client-server setup using two terminal windows to simulate communication between two different machines on the virtual network.

a. Window 1 (The xv6 Client): make qemu is run to start the xv6 guest OS. Inside xv6, the nettest grade command is executed. This program acts as the client, using the newly implemented send and recv system calls to perform a series of network tests (e.g., sending a single packet, performing a ping).
b. Window 2 (The Python Server): On the host machine, the ./nettest.py grade script is run. This Python program acts as the server, listening for incoming packets from xv6. It verifies the content of received packets and sends replies when required by the test (e.g., echoing a packet back for a ping test). This two-way communication, orchestrated by QEMU's networking, validates the entire stack from send on one machine to recv on the other.

5. Conclusion

This lab involved implementing a foundational UDP networking stack for the xv6 operating system, enabling communication between the OS and the host machine. The process required completing a low-level E1000 network driver to handle the physical transmission and reception of packets via DMA rings and building a higher-level network subsystem within the kernel. This subsystem was exposed to user programs through a trio of system calls—bind, send, and recv—which manage ports, send data, and receive data with blocking capabilities. Key challenges included correctly parsing network protocol headers (Ethernet, IP, UDP), handling essential control protocols like ARP, managing packet queues, implementing the crucial sleep/wakeup mechanism for asynchronous I/O, and resolving a critical deadlock between the packet receive and

transmit paths. The result is a functional, albeit minimal, network stack capable of performing real-world tasks like DNS queries, demonstrating the complete data path from a user application, through the kernel, to the hardware, and back again.

OUTPUT: